EgoSim: Egocentric World Simulator for Embodied Interaction Generation

Jinkun Hao, Mingda Jia, Ruiyan Wang, Xihui Liu, Ran Yi, Lizhuang Ma et al.

https://arxiv.org/abs/2604.01001

Research Questions

- How can egocentric world simulators maintain spatial consistency and persistent world state updates across multi-stage interactions?
- How to scale training data for egocentric simulators by automatically extracting aligned scene-interaction pairs from in-the-wild monocular videos?
- Can egocentric simulators trained on human hand data transfer to robotic manipulation tasks?

Core Idea

Existing egocentric world simulators either lack 3D grounding (causing drift) or treat scenes as static (no state persistence). EgoSim solves this by maintaining an explicit 3D point cloud representation that gets updated after each interaction. The key insight is to decouple static scene rendering from dynamic interaction generation, then use the generated observations to update the 3D scene state for the next timestep. This creates a closed-loop simulator where object manipulations (like opening doors) persist in the world state.

Key Formula

Observation Generation

$$O_k = \Pi(S_{k-1}; C_k) + \Delta O(H_k)$$

O_k is generated observation, Pi renders static background from 3D state S_{k-1} and camera C_k, Delta O synthesizes dynamic residuals from hand actions H_k. This decouples static rendering from dynamic interaction generation.

State Update

$$S_k = U(S_{k-1}, O_k)$$

S_k is updated 3D scene state, U is update function, S_{k-1} is previous state, O_k is generated observation. This extracts interaction changes from the observation and applies them to the persistent 3D scene state.

Key Figure

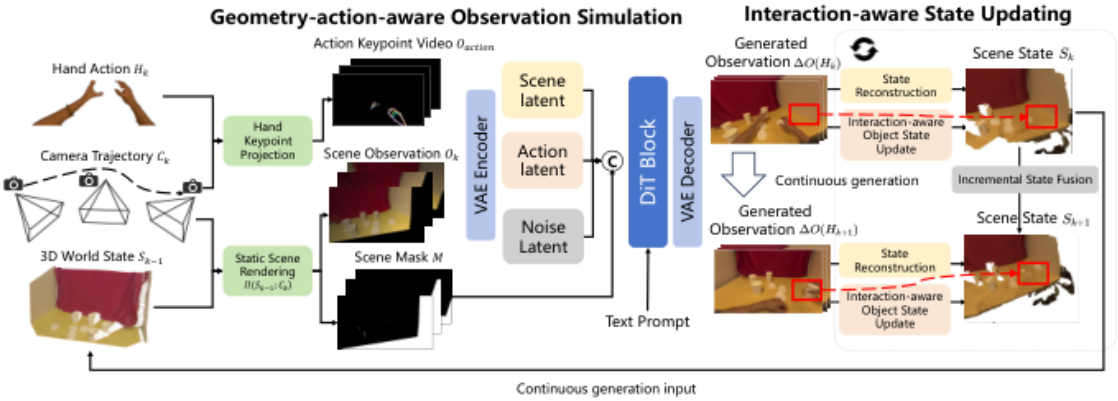

Fig. 2: Overview of EgoSim. Our framework enables continuous egocentric simulation via updatable world modeling. It consists of (1) a Geometry-action-aware Observation Simulation model that synthesizes action-conditioned visual dynamics O_k based on current scene geometry; and (2) an Interaction-aware state updating module that tracks point-based interaction-aware object states from generated observations and integrates them back into the persistent 3D world state to update S_k.

3.2 Geometry-action-aware Observation Simulation

The visual state transition is implemented through a geometry-action-aware video generation model acting as our Observation Simulation model. This model synthesizes the dynamic residuals driven by hand interactions, while ensuring strict adherence to the 3D environment.

Furthermore, to rigorously constrain the generation process, we formulate the conditioning signals by combining explicit scene geometry with action representations. The Scene Observation video O_k provides the background and accurate camera motion cues. Simultaneously, the interaction action is represented by a hand keypoint video O_{action}. Specifically, we extract 3D hand keypoints and project them onto the 2D observation plane using the corresponding camera intrinsics and extrinsics. This perspective projection enforces depth-dependent foreshortening and accurate geometry, anchoring the generated interactions in 3D space. We intentionally employ core joint keypoints rather than dense hand meshes, as keypoints serve as a universal, cross-embodiment action representation that facilitates generalization from human hands to robotic end-effectors. Furthermore, because O_k inevitably contains unrendered regions due to occlusions or incomplete scanning, we introduce a binary mask video M to explicitly indicate these unobserved regions that require visual synthesis.

Training & Inpainting Prior. During training, we operate in the latent space of a pretrained video VAE. The denoising network ϵ_θ, instantiated as a Diffusion Transformer (DiT), processes the concatenated latent $z_{in}^{(t)} = \text{Concat}(z_t, z_{bg}, z_{hand}, M)$ to predict the Gaussian noise ϵ, where all conditions are spatially aligned with the noisy latent z_t:

$$\mathcal{L}_{\text{gen}} = \mathbb{E}_{z_0, t, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\left\| \epsilon - \epsilon_\theta\left(z_{in}^{(t)}, t\right) \right\|_2^2 \right] \tag{3}$$

Figure 2: Shows the complete EgoSim pipeline with both observation simulation and state updating components in closed-loop

operation.

Limitations

- Monocular depth and camera pose estimation can fail in heavily occluded or highly dynamic environments leading to imperfect point cloud initialization
- Limited to manipulation tasks - doesn't handle more complex physical interactions like fluid dynamics or soft body deformation
- Relies on hand keypoint detection which may be unreliable for highly occluded or fast motion scenarios
- State updating currently focuses on rigid object manipulation and may not generalize to complex multi-object interactions